

AUTOCOMPLETION OF CODE BY USING OLLAMA(qwen2.5-coder:7b)-

</>

Lexical Analyzer

Reset

Generate Tokens

Generate NFA

Generate DFA

Source Code

input.js

#include <stdio.h>

Token Rules

+ Add Rule

KEYWORD

→

int|float|if|else

×

IDENTIFIER

→

[a-zA-Z][a-zA-Z0-9]*

×

NUMBER

→

[0-9]+

×

OPERATOR

→

\+|\-|*|\/|=

×

NFA Visualization

VIS.NETWORK

LEGEND

○ Normal State

● Final (Accepting)

--- ε-transition

— Normal transition

Token Output

Live generated

TYPE

LEXEME

No tokens generated yet.

</>

Lexical Analyzer

Reset

Generate Tokens

Generate NFA

Generate DFA

Source Code

input.js

#include <stdio.h>
int main() {
int x=10;
int y=20;
int sum= x + y;
printf("Sum: %d\n", sum);

Token Rules

+ Add Rule

KEYWORD

→

int|float|if|else

×

IDENTIFIER

→

[a-zA-Z][a-zA-Z0-9]*

×

NUMBER

→

[0-9]+

×

OPERATOR

→

\+|\-|*|\/|=

×

NFA Visualization

VIS.NETWORK

LEGEND

○ Normal State

● Final (Accepting)

--- ε-transition

— Normal transition

Token Output

Live generated

TYPE

LEXEME

No tokens generated yet.

2.) LexiAI Assistant(Implemented By Gemini Api key)-

The screenshot shows the Lexical Analyzer interface with the Lexi AI Assistant chat window open. The chat window displays the source code and asks for regex rules. The interface includes buttons for Reset, Generate Tokens, Generate NFA, and Generate DFA.

Source Code:

```
#include <stdio.h>
int main() {
    int x=10;
    int y=20;
    int sum= x + y;
    printf("Sum: %d\n", sum);
    return 0;
};
```

Token Rules:

- KEYWORD → `int|float|if|else`
- IDENTIFIER → `[a-zA-Z][a-zA-Z0-9]*`
- NUMBER → `[0-9]+`
- OPERATOR → `\+|\-|\*|\/|=`

Lexi AI Assistant Chat:

Hi there! I would love to help you write the regular expressions for this C code snippet. Breakdowns like this are the first step in building a lexical analyzer (lexer) for a compiler!

To tokenize this code, we can group the characters into different token types. Here are the regex rules for each

Regex for Float? Hexadecimal pattern? String matching?

Ask Lexi about regex...

3.) NFA DFA VISUALAZIATION-

The screenshot shows the Lexical Analyzer interface with the DFA Visualization window open. The chat window displays the source code and asks for regex rules. The interface includes buttons for Reset, Generate Tokens, Generate NFA, and Generate DFA.

Source Code:

```
#include <stdio.h>
int main() {
    int x=10;
    int y=20;
    int sum= x + y;
    printf("Sum: %d\n", sum);
    return 0;
};
```

Token Rules:

- Integer Constans → `\b[0-9]+\b`
- String Literals → `"[A]"`
- Operators → `[+=]`
- Punctuation and → `[{};,:]`

DFA Visualization:

Legend:

- Normal State
- Final (Accepting)
- ε-transition
- Normal transition

Token Output:

TYPE	LEXEME
Punctuation and Delimiters	{
Keywords	int

